

Intern Mini Project

Project: E-Commerce Order Analytics System

Duration: 3-4 weeks

Skills Tested: Python, SQL, Problem Solving

Background

You are joining a company that processes online orders. The raw data comes from multiple sources and is messy. Your job is to clean it, transform it, and generate business reports.

This project has following **phase**:

- **Phase:** Build everything using Python and SQL (local environment)
-

The Data

You will create 4 CSV files with sample data (at least 500 rows each):

1. orders.csv

order_id, customer_id, order_date, status, region_code

- status can be: PLACED, SHIPPED, DELIVERED, CANCELLED, RETURNED
- order_date format: YYYY-MM-DD HH:MM:SS
- Some rows have missing customer_id (marked as NULL or empty)

2. order_items.csv

item_id, order_id, product_id, quantity, unit_price, discount_percent

- discount_percent is between 0 and 100
- Some rows have negative quantity (these are returns)

3. products.csv

```
product_id, product_name, category, subcategory, cost_price
```

- category examples: Electronics, Clothing, Home, Books

4. customers.csv

```
customer_id, customer_name, email, registration_date, customer_type
```

- customer_type: REGULAR, PREMIUM, VIP
-

PHASE 1: Python & SQL (Local Environment)

Part 1: Data Generation (Python)

Write a Python script to generate the above 4 CSV files with realistic fake data.

Requirements:

- Data should have intentional issues:
 - 5% of orders should have NULL customer_id
 - 3% of order_items should have negative quantity
 - Some orders should have order_date in wrong format (DD-MM-YYYY)
 - Some product names should have extra spaces or mixed case
 - 2% of emails should be invalid (missing @ or domain)

Think about: How will you ensure order_id in order_items actually exists in orders table?

Part 2: Data Cleaning (Python)

Write Python functions (pandas allowed) to:

1. **clean_orders()** - Fix date formats, handle NULL customer_ids
2. **clean_products()** - Normalize product names (trim spaces, title case)

3. **validate_emails()** - Return list of customer_ids with invalid emails
4. **check_referential_integrity()** - Find order_items that reference non-existent orders

Output: Cleaned CSV files + a report of all issues found

Part 3: SQL Analysis

Using SQLite (or any SQL database), load your cleaned data and write queries for:

Basic Queries

1. Total revenue per category (revenue = quantity × unit_price × (1 - discount_percent/100))
2. Top 10 customers by total order value
3. Month-wise order count for the last 12 months

Intermediate Queries

4. Find customers who placed orders but never had any item delivered
5. Products that were ordered but had more returns than purchases
6. Calculate the return rate (returned items / total items) per category

Advanced Queries (Window Functions, CTEs, Subqueries)

7. Running Totals with Window Functions

Calculate running total of revenue per region, ordered by date.
Show: region_code, order_date, daily_revenue, running_total

8. Ranking with DENSE_RANK

For each category, rank products by total revenue.
Show: category, product_name, total_revenue, rank_in_category
Products with same revenue should have same rank.

9. LAG/LEAD Analysis

For each customer, calculate days between consecutive orders.
Show: customer_id, order_date, previous_order_date, days_gap
Flag customers with average gap > 30 days as "At Risk"

10. CTE with Multiple Levels

Using CTEs, find:

- First, calculate monthly revenue per customer
- Then, categorize customers: 'High' (>10000), 'Medium' (5000-10000), 'Low' (<5000)
- Finally, show count of customers in each category per month

11. NTILE for Segmentation

Divide customers into 4 quartiles based on total lifetime value.

Show: customer_id, total_value, quartile, quartile_label (Platinum/Gold/Silver/Bronze)

12. Year-over-Year Comparison

Compare each month's revenue with same month previous year.

Show: year, month, revenue, prev_year_revenue, yoy_growth_percent

Handle cases where previous year data doesn't exist.

13. First/Last Value Analysis

For each customer, show their first purchased category and most recent purchased category.

Flag if they are different (category_shift = 'Yes'/'No')

14. Cumulative Distribution

Calculate what percentage of total revenue comes from top N% of customers.

Show: customer_id, revenue, cumulative_revenue, cumulative_percent

15. Complex CTE: Cohort Analysis

Group customers by their registration month (cohort).

For each cohort, calculate:

- How many ordered in month 0 (registration month)
- How many ordered in month 1, month 2, month 3
- Retention rate for each month

16. Self-Join with Window Function

Find products frequently bought together.

Show: product_a, product_b, times_bought_together

Exclude same product pairs and duplicates (A-B and B-A should appear once)

Part 4: Python + SQL Integration

Build a simple command-line tool that:

1. Takes user input for report type (daily/weekly/monthly)
2. Takes date range as input
3. Connects to SQLite database
4. Generates a summary report showing:
 - Total orders, revenue, unique customers
 - Top 3 products
 - Comparison with previous period (% change)

No external libraries except sqlite3

Part 5: Edge Case Handling

Write test cases (as Python functions) that verify:

1. What happens when order_items has an order_id not in orders?
 2. What happens when discount_percent > 100?
 3. What happens when quantity is 0?
 4. What happens when order_date is in the future?
-